

# Bitácoras Sistemas Embebidos Primer Corte

Julian Díaz Cortina  
*Ingeniería Mecatrónica*  
*Universidad Santo Tomás*  
Bucaramanga, Colombia

juliandavid.diaz@ustabuca.edu.co

Fabian Ruiz Mateus  
*Ingeniería Mecatrónica*  
*Universidad Santo Tomás*  
Bucaramanga, Colombia

fabianaugusto.ruiz@ustabuca.edu.co

**Abstract**—This document is a template in Spanish for writing articles in LaTeX using the IEEE format. It briefly explains the objective of the work, the general methodology, and the main results obtained.

**Index Terms**—LaTeX, IEEEtran, templates, figures, tables, bibliography.

## RESUMEN

Este documento es una plantilla en español para escribir artículos en LaTeX con el formato IEEE. Aquí se explica brevemente el objetivo del trabajo, la metodología general y el resultado principal.

## PALABRAS CLAVE

LaTeX, IEEEtran, plantillas, figuras, tablas, bibliografía.

## I. INTRODUCCIÓN

En los artículos originales enviados a la revista IEEE, la introducción debe presentar de forma concisa el contexto y la relevancia del problema abordado, describiendo brevemente el área de estudio y su importancia en el campo de la ingeniería o tecnología; incluir un resumen del estado del arte con referencia a los trabajos más relevantes y recientes, destacando las limitaciones o vacíos existentes que motivan la investigación; exponer con claridad el objetivo y la principal contribución del trabajo, indicando qué se propone y en qué se diferencia o mejora respecto a lo ya publicado; y, opcionalmente, cerrar con una breve descripción de la estructura del artículo para guiar al lector en el desarrollo del contenido.

La escritura académica en LaTeX ofrece control tipográfico y reproducibilidad. Esta plantilla en español muestra, con ejemplos mínimos, cómo estructurar un artículo usando IEEEtran: figuras, tablas, ecuaciones y bibliografía. Úsala para practicar el flujo básico de edición y compilación.

El resto del documento se organiza así: en la Sección II se describe un ejemplo de desarrollo experimental simplificado; en la Sección III se explica la metodología con ecuaciones de ejemplo; en la Sección VII se muestran tablas y figuras de ejemplo; y en la Sección VIII se listan conclusiones y trabajo futuro.

LaTeX facilita gestionar referencias con BibTeX, por ejemplo [? ?]. También puedes usar estilos numéricos compatibles con IEEE. Las citas deben registrarse en el archivo `reference.bib`

Diversos trabajos describen técnicas de análisis espectral y aprendizaje automático aplicadas a señales [? ?]. Ajusta esta sección a tu tema específico.

## II. MARCO REFERENCIAL

### A. Actividad 1

1) *Caso de estudio*: Se desea diseñar un sistema basado en microcontrolador que:

- Lea un sensor cada 10 ms.
- Controle un motor según la lectura.
- Envíe datos por comunicación serial cada 100 ms.
- Atienda un botón de emergencia que debe detener todo inmediatamente.
- Muestre información en una pantalla.

• **¿Cómo implementar esto usando solo un while?:** La respuesta de esta pregunta genera como resultado el diagrama de flujo que se encuentra en la figura 1.

• **¿Qué problemas podrían aparecer?:** Al ser un while con dependencia doble, interna y distinta del tiempo; se puede estar presentando retrasos de lectura, pues la lectura lineal debe esperar a, por ejemplo, enviar el paquete de datos por el puerto serial para continuar con el proceso. Esto mismo trae problemas de latencia y de estabilidad, pues en el caso que se desee agregar mas tareas, el control del tiempo se verá aún mas afectado.

• **¿Qué tarea sería más crítica?:** En este caso desarrollado, se entiende que la tarea más crítica sería toda la que conlleve al botón de emergencia, pues estamos hablando de factores de seguridad humana y estos deben ser considerados de manera prioritaria.

• **¿Cómo garantizarías tiempos de respuesta?:** En una primera parte, se deberían independizar dos elementos claves: La parada de emergencia y el timer, pues estos deben ser independientes de una revisión ocasional dentro del while. Una manera de hacerlo es generando interrupciones; asegurando una baja latencia y por lo tanto (en el caso de la parada de emergencia) un menor tiempo de reacción. Por parte del Timer; considero que se debería tratar como variable independiente donde dicha variable sea la única que gestione la lectura y el envío de datos con

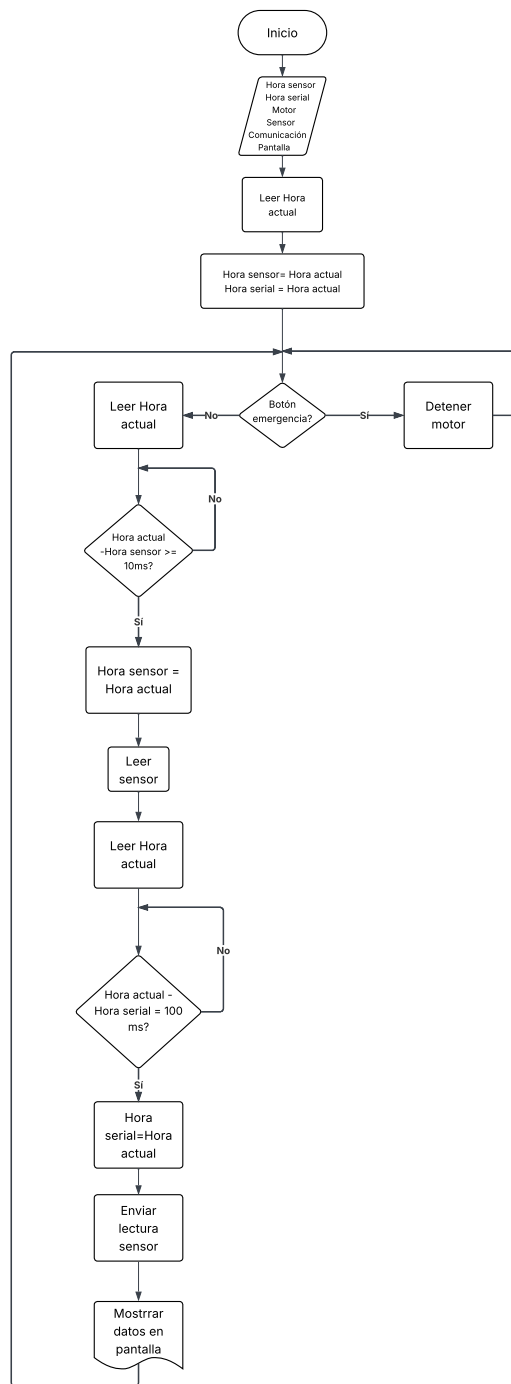


Figure 1. Diagrama de flujo de la implementación del while

el fin de asegurar un "espacio de memoria" que realmente cumpla los tiempos establecidos.

## 2) Trabajo de consulta:

- **Interrupciones:** Las interrupciones son mecanismos que permiten a un microcontrolador suspender temporalmente la ejecución del programa para atender un evento inmediato. Cuando ocurre dicho evento, el hardware guarda el

estado del procesador, ejecuta una Rutina de Servicio de Interrupción (*Interrupt Service Routine-ISR*) y luego restaura el estado para continuar la ejecución normal.

Su función principal es evitar el *polling* (método en el que el procesador revisa continuamente si ocurrió un evento, en lugar de esperar a que una interrupción lo notifique) constante y mejorar el tiempo de respuesta ante eventos internos o externos, siendo esenciales en sistemas de tiempo real. [?] No obstante, pueden generar condiciones de carrera, aumento de latencia si la ISR es extensa y problemas de memoria de pila, por lo que deben implementarse de forma breve y controlada. [?]

- **Planificación y Scheduler:** El scheduler (planificador) es el componente del sistema operativo o del software de control que decide qué tarea se ejecuta y en qué momento. En sistemas embebidos con multitarea, su función es administrar el uso del procesador cuando existen varias tareas que requieren ejecución, garantizando orden y control en el acceso al CPU (*Central Processing Unit*). [?] La prioridad es el nivel de importancia asignado a cada tarea. Determina cuál debe ejecutarse primero cuando varias están listas al mismo tiempo... Una tarea con mayor prioridad desplaza a otra de menor prioridad si el sistema lo permite. La planificación preventiva (preemptiva) es aquella en la que el scheduler puede interrumpir una tarea en ejecución para asignar el procesador a otra de mayor prioridad. En cambio, la planificación cooperativa requiere que la tarea en ejecución ceda voluntariamente el control del procesador, por lo que el cambio de tarea ocurre solo cuando esta finaliza o libera el CPU. [?]

- **Multitarea:** La concurrencia es la capacidad de un sistema para gestionar múltiples tareas que progresan en intervalos de tiempo superpuestos. No implica necesariamente que todas se ejecuten exactamente al mismo tiempo, sino que el sistema organiza su ejecución de manera que todas avancen sin bloquearse mutuamente. [?] La multitarea real ocurre cuando varias tareas se ejecutan simultáneamente en diferentes núcleos de procesamiento. En cambio, la multitarea simulada se presenta en sistemas con un solo núcleo, donde el procesador alterna rápidamente entre tareas mediante un scheduler, dando la impresión de ejecución simultánea aunque solo una tarea se ejecuta en cada instante.

En sistemas embebidos, ejemplos de multitarea incluyen: un microcontrolador que controla un motor mientras lee sensores y transmite datos por comunicación serial; un sistema IoT (*Internet of Things*) que adquiere datos, los procesa y los envía a la nube; o un dispositivo médico que monitorea señales mientras actualiza una pantalla. En todos estos casos, las tareas comparten el procesador y requieren coordinación adecuada. [?]

- **Sincronización:** Una condición de carrera es una situación que ocurre cuando dos o más tareas acceden

y modifican un mismo recurso o variable compartida de manera simultánea, y el resultado final depende del orden impredecible en que se ejecuten [? ]. Esto puede generar inconsistencias o corrupción de datos si no existen mecanismos de control adecuados.

Los semáforos y los mutex son mecanismos de sincronización utilizados para coordinar el acceso a recursos compartidos. Un semáforo es una variable de control que regula la disponibilidad de un recurso y puede permitir uno o varios accesos según su tipo. Un mutex (mutual exclusion) es un mecanismo específico que garantiza que solo una tarea a la vez pueda acceder a un recurso, asegurando exclusión mutua.

Una sección crítica es el fragmento de código en el que se accede a un recurso compartido y que, por lo tanto, debe ejecutarse de forma exclusiva para evitar interferencias entre tareas. Para protegerla, se emplean mecanismos como mutex, semáforos o des-habilitación temporal de interrupciones. [? ]

### III. PRACTICAS

#### A. Reto 1

Diseñar un sistema que permita activar una cerradura electrónica si se cumple una condición lógica (contraseña digital o combinación de botones). Requisitos obligatorios:

- Entrada digital (botones o teclado)
- Salida: LED indicador + actuador (servo o relé)
- Manejo de intentos fallidos
- Temporizador de bloqueo tras múltiples errores
- Modelado opcional como máquina de estados

##### 1) *Análisis del problema:*

- **Identificación de entradas y salidas:** Según los requisitos obligatorios del sistema, se identifica la necesidad de tener una entrada digital correspondiente, en este caso; a tres botones que ejecutarán la secuencia combinacional para la apertura de la cerradura. En lo que corresponde a la salida, se usa un Diodo LED como indicador lumínico de apertura de la cerradura que será representada por el funcionamiento de un servomotor.
- **Restricciones técnicas:** El sistema presenta restricciones eléctricas relacionadas con los niveles de tensión, corriente máxima de los pines GPIO y los requerimientos de alimentación del actuador.
  - El sistema presenta unos límites eléctricos correspondientes a sus 3 elementos principales, ESP32, Servomotor y Led. El microcontrolador ESP32 opera con niveles lógicos de 3.3[V] en sus pines GPIO que pueden suministrar corrientes limitadas (12 mA recomendados [? ]) . El servomotor requiere una alimentación de aproximadamente 5[V] y puede generar picos de corriente durante su operación [? ]. El LED necesita una resistencia limitadora para controlar y así mismo evitar el sobre consumo.
  - El sistema debe controlar tiempos específicos, como la duración de apertura del servo y el periodo de

bloqueo tras múltiples intentos fallidos. Por lo tanto, el funcionamiento depende de una gestión precisa del tiempo para garantizar un comportamiento adecuado.

- Las entradas mediante pulsadores pueden presentar rebote mecánico, lo que puede provocar lecturas inestables si no se considera esta condición en el comportamiento del sistema.

- **Variables Involucradas:** Para la implementación del algoritmo de control se definen diversas variables que permiten gestionar el funcionamiento del sistema. Entre ellas se encuentra un arreglo que almacena la contraseña correcta, otro arreglo que almacena la combinación ingresada por el usuario, una variable entera que registra el número de intentos fallidos y una constante que define el número máximo permitido antes de activar el bloqueo. Adicionalmente, se utilizan variables booleanas para indicar si el sistema se encuentra bloqueado y variables relacionadas con temporización para controlar la duración del bloqueo.

2) **Flujograma del sistema:** Se encuentra en la figura 2

##### 3) *Justificación técnica del microcontrolador seleccionado:*

- **Número de pines requeridos:** El sistema requiere tres pines de entrada digital para los botones utilizados en la combinación de acceso. Adicionalmente, se requieren dos pines de salida para los LED indicadores de estado (acceso autorizado y acceso denegado) y un pin de salida adicional para el control del actuador de la cerradura (servo motor o relé). En total se requieren aproximadamente seis pines GPIO. El ESP32 dispone de más de 30 pines de entrada/salida programables, por lo que cumple ampliamente con este requerimiento.
- **Capacidad de procesamiento:** El microcontrolador cuenta con un procesador de doble núcleo basado en arquitectura Tensilica LX6 con frecuencia de operación de hasta 240 MHz, lo cual es suficiente para ejecutar la lógica de control del sistema, realizar la lectura de entradas digitales, gestionar temporizadores y generar señales PWM necesarias para el control de actuadores como servomotores. Su memoria RAM interna también permite implementar estructuras de datos para manejo de estados e intentos de autenticación.
- **Consumo energético:** El ESP32 presenta un consumo aproximado entre 80 mA y 240 mA en modo activo dependiendo de la carga de procesamiento, y puede reducir su consumo a valores del orden de microamperios cuando se utilizan modos de bajo consumo como deep sleep. Esto resulta adecuado para sistemas de control que permanecen largos periodos en estado de espera.
- **Esquema de conexión:** Se encuentra en la imagen 3

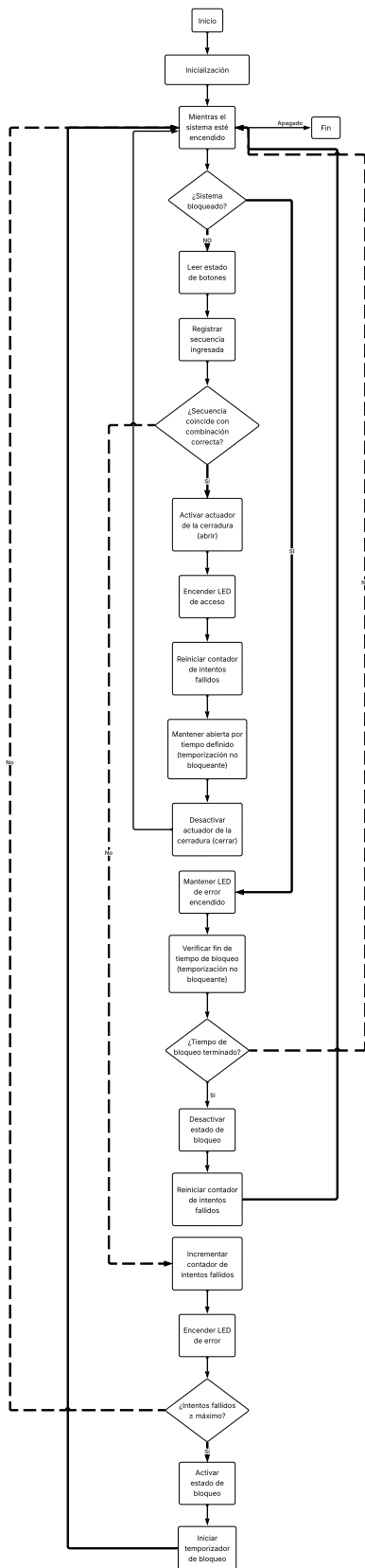


Figure 2. Flujograma reto 1

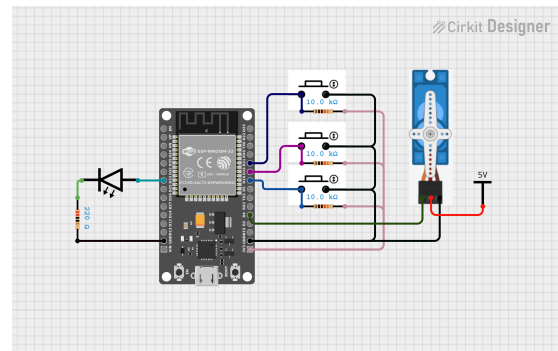


Figure 3. Diagrama de conexión reto 1

- **Componentes auxiliares** Para el correcto funcionamiento del sistema se requieren resistencias limitadoras de corriente para los LED (típicamente de 220 Ohm a 330 Ohm), resistencias de polarización para los botones (aproximadamente 10 kOhm), una fuente de alimentación estable para el microcontrolador y el actuador, y en caso de utilizar relé, un transistor driver con diodo de protección para evitar daños por voltajes inducidos. Estos componentes garantizan estabilidad eléctrica, protección de los dispositivos y operación confiable del sistema.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-A

## B. Reto 2

Desarrollar un sistema que supervise una variable física y active alarmas según tres niveles de operación: normal, advertencia y crítico. Requisitos obligatorios:

- Definición clara de umbrales
- Implementación de histéresis para evitar inestabilidad
- Indicadores visuales o sonoros

### 1) Análisis del problema:

- **Identificación de entradas y salidas:** El sistema requiere una entrada correspondiente a la medición de una variable física que permita evaluar el estado del entorno supervisado. Para efectos de este diseño se selecciona la temperatura como variable de monitoreo, ya que constituye una de las variables más comunes en procesos industriales, sistemas electrónicos y aplicaciones de supervisión ambiental.

La medición de la temperatura será realizada mediante un sensor de temperatura analógico, el cual entrega una señal proporcional a la magnitud física medida. Esta señal será adquirida por el microcontrolador a través de uno de sus canales de conversión analógico-digital (ADC), permitiendo así transformar la señal analógica en un valor digital interpretable por el sistema.

En lo que corresponde a las salidas, el sistema contará con indicadores visuales y sonoros que permitirán identificar el estado operativo del sistema de monitoreo. Para ello se emplearán tres diodos LED que representarán los estados de operación normal, advertencia y condición crítica

respectivamente. De manera adicional se incorporará una alarma sonora mediante un buzzer, el cual se activará únicamente cuando el sistema detecte que la variable monitoreada ha alcanzado el nivel crítico.

• **Restricciones técnicas:**

- El sensor de temperatura seleccionado entrega una señal analógica que debe ser interpretada por el microcontrolador [? ]. Por lo tanto, el dispositivo de procesamiento debe contar con un convertidor analógico–digital con suficiente resolución para permitir una medición adecuada de la variable física.
- Los diodos LED empleados como indicadores visuales requieren la inclusión de resistencias limitadoras de corriente, ya que los pines GPIO del microcontrolador presentan límites de corriente que no deben ser superados para evitar daños en el dispositivo.
- En el caso del buzzer o alarma sonora, es necesario considerar que algunos dispositivos pueden requerir corrientes superiores a las que puede suministrar directamente un pin del microcontrolador[? ]. En estas condiciones puede ser necesario incorporar una etapa de conmutación basada en transistor, que permita manejar la carga sin comprometer la integridad del microcontrolador.
- Una restricción fundamental en este tipo de sistemas corresponde a la inestabilidad que puede presentarse cerca de los umbrales de decisión. Cuando la variable medida se encuentra próxima al valor límite entre dos estados, pequeñas fluctuaciones pueden provocar cambios continuos entre estados consecutivos. Para evitar este comportamiento se implementa un mecanismo de histéresis, el cual introduce un margen de diferencia entre los valores de activación y desactivación de cada estado [? ].

- **Variables Involucradas:** Variable que almacena el valor digital leído del convertidor analógico–digital correspondiente al sensor de temperatura. Variables que definen los umbrales del sistema para los estados normal, advertencia y crítico. Variable que indica el estado actual del sistema para activar los indicadores correspondientes. Variable que establece el margen de histéresis para evitar cambios repetitivos entre estados. Variables auxiliares para controlar el tiempo de muestreo y la frecuencia de lectura del sensor.

2) **Flujograma del sistema:** Se encuentra en la figura 4.

3) **Justificación técnica del microcontrolador seleccionado:**

- **Número de pines requeridos.** El sistema requiere al menos un pin de entrada analógica para la lectura del sensor que mide la variable física supervisada. Adicionalmente, se requieren tres pines de salida digital para los indicadores de estado correspondientes a los niveles de operación: normal, advertencia y crítico. También puede incorporarse un pin adicional para activar una

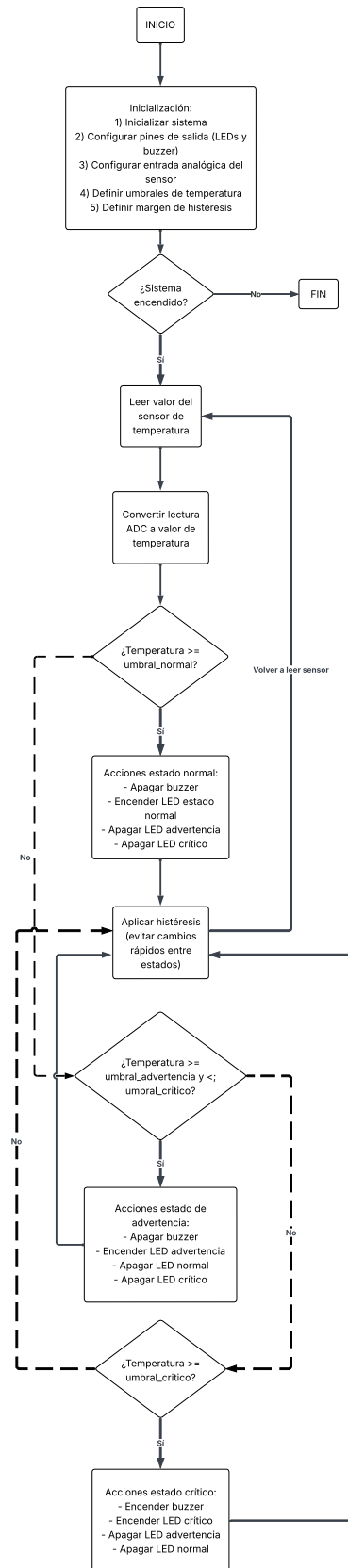


Figure 4. Flujograma reto 2

alarma sonora mediante un buzzer. En total, el sistema requiere aproximadamente cinco pines de entrada/salida. El microcontrolador ESP32 dispone de múltiples pines GPIO y varios canales ADC de 12 bits [? ], lo que permite realizar la adquisición de la señal del sensor con suficiente resolución.

- **Capacidad de procesamiento.** El ESP32 cuenta con un procesador de doble núcleo con frecuencia de operación de hasta 240 MHz [? ], lo cual permite ejecutar el algoritmo de monitoreo en tiempo real, realizar la conversión analógica-digital de la señal del sensor y aplicar la lógica de comparación con los umbrales definidos. Además, su capacidad de procesamiento permite implementar histéresis en los niveles de alarma para evitar conmutaciones rápidas e inestables cuando la variable medida se encuentra cercana a los valores límite [? ].
- **Consumo energético.** El dispositivo presenta un consumo aproximado entre 80 mA y 240 mA en modo activo, dependiendo de la carga de procesamiento y los periféricos utilizados. Asimismo, dispone de modos de bajo consumo que reducen significativamente la corriente cuando el sistema se encuentra en espera. Esto permite su utilización en sistemas de monitoreo continuo sin requerir un consumo energético elevado.
- **Esquema de conexión.** Se encuentra en la imagen 5

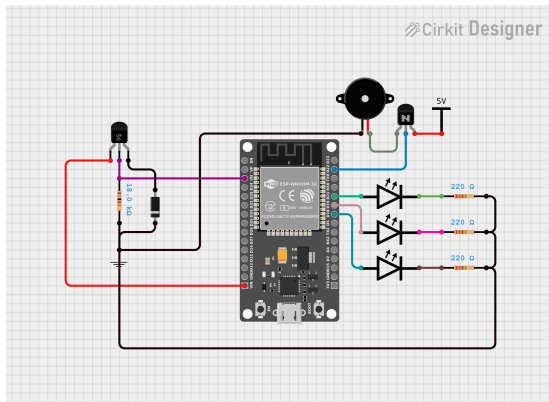


Figure 5. Diagrama de conexión reto 2

- **Componentes auxiliares** El sistema requiere resistencias limitadoras de corriente para los LED indicadores (220 Ohm a 330 Ohm), resistencias de polarización en caso de utilizar sensores analógicos que lo requieran, y una fuente de alimentación estable para el microcontrolador y los dispositivos de salida. Si se emplea un buzzer activo o una alarma de mayor potencia, puede ser necesario utilizar un transistor driver para manejar la corriente requerida.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-B

### C. Reto 3

Diseñar un sistema de riego automático para cultivo de tomate. El sistema debe: Monitorear la humedad del suelo,

Activar riego cuando la humedad esté por debajo de un umbral, Detener el riego cuando se alcance nivel adecuado, Evitar inestabilidad mediante histéresis, Funcionar como sistema secuencial autónomo e Implemente temporización no bloqueante.

#### 1) **Análisis del problema:**

- **Identificación de entradas y salidas:** La entrada principal es un sensor de humedad del suelo conectado a una entrada analógica del microcontrolador. La salida corresponde al actuador de riego (bomba o válvula solenoide) controlado mediante un relé o transistor de potencia. Opcionalmente se puede incluir un LED indicador de estado del sistema.

#### • **Restricciones técnicas:**

- El microcontrolador debe disponer de al menos una entrada analógica (ADC) para la lectura del sensor de humedad.
- Debe existir una salida digital capaz de controlar un relé o transistor para activar la bomba o válvula de riego.
- El sistema debe operar con niveles de voltaje compatibles (3.3 V o 5 V) entre microcontrolador, sensor y etapa de potencia. [? ]
- El control del riego debe implementar histéresis mediante dos umbrales de humedad para evitar conmutaciones rápidas.
- El software debe emplear temporización no bloqueante (por ejemplo mediante contadores de tiempo) para permitir lecturas periódicas del sensor.
- **Variables involucradas:** Las variables principales son el valor de humedad del suelo medido por el sensor, el umbral inferior de activación del riego, el umbral superior de desactivación, el estado del sistema (riego activo o inactivo) y las variables de temporización utilizadas para el muestreo periódico del sensor.

2) **Flujograma del sistema :** Se encuentra en la figura 6

3) **Justificación técnica del microcontrolador seleccionado:**

- **Número de pines requeridos:** El sistema requiere 1 entrada analógica para el sensor de humedad del suelo y 1 salida digital para controlar el relé o transistor que activa la bomba de riego. Opcionalmente se puede utilizar 1 salida digital adicional para un LED indicador. El ESP32 dispone de más de 30 pines GPIO, incluyendo múltiples entradas ADC, por lo que satisface ampliamente los requerimientos del sistema.
- **Capacidad de procesamiento:** El ESP32 integra un procesador dual-core de hasta 240 MHz [? ], suficiente para ejecutar el algoritmo de control secuencial, realizar lecturas periódicas del sensor mediante ADC, aplicar histéresis entre umbrales de humedad y gestionar temporización no bloqueante para el monitoreo continuo del sistema.
- **Consumo energético:** El microcontrolador opera a 3.3 V y presenta un consumo aproximado de 80–240 mA en

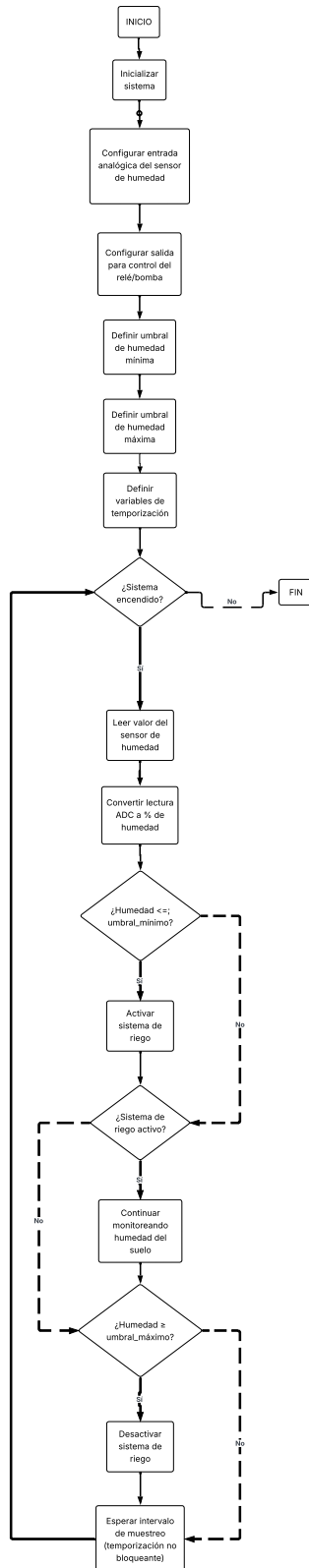


Figure 6. Flujograma reto 3

operación [? ], con modos de bajo consumo disponibles. Esto permite su integración en sistemas agrícolas automatizados con alimentación estable o mediante fuentes reguladas.

- **Esquema de conexión (diagrama eléctrico o esquemático):** Se encuentra en la imagen 7

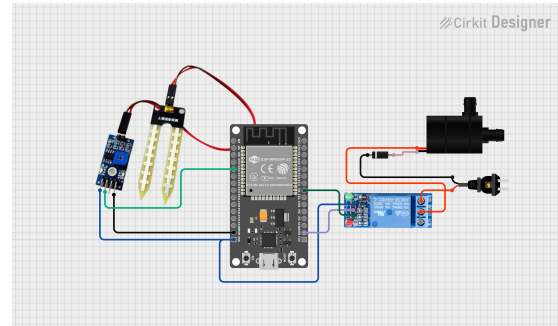


Figure 7. Diagrama de conexión reto 3

- **Componentes auxiliares** El sistema requiere un sensor de humedad del suelo, un módulo relé o MOSFET de potencia, resistencias de polarización para el control del transistor, diodo de protección, una bomba de agua o válvula solenoide para el riego y opcionalmente LEDs indicadores para mostrar el estado del sistema.

4) **Código documentado:** El código respectivo se encuentra en el anexo IX-C

#### IV. PROTOCOLOS DE COMUNICACIÓN

Los protocolos de comunicación permiten el intercambio de datos entre dispositivos dentro de un sistema embebido. Estos pueden ser de tipo serial o inalámbrico, dependiendo del medio de transmisión.

##### A. Comunicación Serial

La comunicación serial se basa en la transmisión secuencial de datos bit a bit a través de un canal, lo que permite reducir el cableado y mejorar la eficiencia del hardware.

- Transmisión secuencial de información
- Reducción del cableado
- Mayor eficiencia en el diseño
- Conexión entre microcontroladores y periféricos

Dentro de la comunicación serial se destacan los siguientes protocolos:

##### 1) UART (Universal Asynchronous Receiver-Transmitter):

Es un protocolo de comunicación asíncrono que no utiliza señal de reloj compartida.

- Comunicación punto a punto
- Uso de tramas de datos
- Configuración mediante baudios, bits de datos, paridad y bits de parada
- Solo requiere dos líneas: TX y RX (más GND)

##### Ventajas:

- Simplicidad

- Bajo costo
- Comunicación full-duplex
- Mayor alcance comparado con otros protocolos seriales

2) **I2C (Inter-Integrated Circuit)**: Es un protocolo síncrono que permite la conexión de múltiples dispositivos utilizando solo dos líneas.

- Líneas: SDA (datos) y SCL (reloj)
- Permite múltiples dispositivos en el mismo bus
- Maneja direccionamiento (7 y 10 bits)
- Requiere resistencias pull-up

#### Ventajas:

- Reduce el número de conexiones
- Facilita la interconexión de varios periféricos

3) **SPI (Serial Peripheral Interface)**: Es un protocolo síncrono de alta velocidad basado en una arquitectura maestro-esclavo.

- Líneas: MOSI, MISO, SCK y SS/CS
- Comunicación full-duplex
- Alta velocidad de transmisión
- Sincronización mediante señal de reloj

#### Ventajas:

- Alta velocidad
- Precisión en la transmisión de datos

#### 4) Comparación de Protocolos:

| Característica | UART      | I2C       | SPI                 |
|----------------|-----------|-----------|---------------------|
| Tipo           | Asíncrono | Síncrono  | Síncrono            |
| Líneas         | TX, RX    | SDA, SCL  | MOSI, MISO, SCK, SS |
| Reloj          | No        | Sí        | Sí                  |
| Dispositivos   | 2         | Múltiples | Maestro-esclavos    |
| Velocidad      | Media     | Media     | Alta                |
| Complejidad    | Baja      | Media     | Media               |

#### 5) Criterios de Selección:

- Para alta velocidad: SPI
- Para múltiples dispositivos con pocas líneas: I2C
- Para comunicación simple punto a punto: UART

### B. Comunicación Inalámbrica

Los protocolos inalámbricos son fundamentales en aplicaciones IoT y permiten la comunicación sin necesidad de conexiones físicas.

- Bluetooth Low Energy (BLE): bajo consumo y corto alcance
- Wi-Fi: alta tasa de datos e integración a internet
- Zigbee: redes malladas de bajo consumo
- Thread: comunicación en malla basada en IPv6
- LoRaWAN: largo alcance y bajo consumo
- NB-IoT: comunicación celular para IoT

#### 1) Principales protocolos inalámbricos:

a) **Wi-Fi**: Protocolo diseñado inicialmente para redes de computadoras y acceso a internet. Actualmente es ampliamente utilizado en dispositivos IoT con suficiente disponibilidad energética.

- Alcance: ~100 m
- Velocidad: Alta (hasta Gbps)
- Uso: Cámaras, ESP32, interfaces HMI

b) **Bluetooth y Bluetooth Low Energy (BLE)**: Bluetooth es un estándar de corto alcance, mientras que BLE está optimizado para bajo consumo energético, ideal para dispositivos alimentados por batería.

- Alcance: 10–100 m
- Velocidad: Media (1–3 Mbps)
- Uso: Wearables, sensores cercanos

c) **ZigBee**: Protocolo orientado a redes de sensores distribuidos con bajo consumo energético, utilizando topologías malladas.

- Alcance: ~100 m
- Velocidad: Baja (250 kbps)
- Uso: Domótica, automatización industrial

d) **Thread**: Protocolo de red mallada basado en IPv6, diseñado para IoT, con capacidad de auto-reparación y bajo consumo energético.

- Red mallada segura
- Optimizado para dispositivos de baja potencia
- Uso: Hogares inteligentes y edificios

e) **LoRaWAN**: Protocolo de largo alcance y bajo consumo, ideal para aplicaciones donde los dispositivos están muy alejados entre sí.

- Alcance: 2–15 km
- Velocidad: Muy baja
- Uso: Agricultura, monitoreo ambiental, ciudades inteligentes

| Tecnología    | Alcance  | Velocidad | Aplicación                       |
|---------------|----------|-----------|----------------------------------|
| Wi-Fi         | ~100 m   | Alta      | Internet, video, IoT con energía |
| Bluetooth/BLE | 10–100 m | Media     | Sensores, wearables              |
| ZigBee        | ~100 m   | Baja      | Domótica                         |
| LoRaWAN       | 2–15 km  | Muy baja  | Monitoreo remoto                 |

Table 1

COMPARACIÓN DE PROTOCOLOS INALÁMBRICOS

2) **Criterios de selección**: La elección del protocolo inalámbrico debe considerar:

- **Alcance**: Distancia entre dispositivos
- **Consumo energético**: Especialmente en dispositivos con batería
- **Velocidad de transmisión**: Cantidad de datos a enviar
- **Aplicación**: Tipo de sistema (industrial, doméstico, urbano)

En general:

- Wi-Fi: cuando se requiere alta velocidad
- BLE: cuando se necesita bajo consumo
- ZigBee/Thread: redes de múltiples dispositivos
- LoRaWAN: comunicación a larga distancia

3) **Importancia en IoT**: Los protocolos inalámbricos son la base del IoT, ya que permiten conectar sensores, dispositivos y sistemas a internet sin infraestructura cableada. Su correcta selección impacta directamente en el rendimiento, eficiencia y escalabilidad del sistema.

## V. INTERNET TRADITIONAL VERSUS IOT

El internet tradicional se centra principalmente en la interacción entre usuarios y servicios digitales, donde el flujo de información incluye datos como texto, imágenes, audio y video. En este contexto, el usuario es el actor principal y el objetivo es el intercambio de información mediante plataformas web.

En contraste, el Internet de las Cosas (IoT) amplía este paradigma al integrar objetos físicos capaces de captar variables del entorno, procesarlas y comunicarlas a través de la red. El cambio fundamental radica en que el entorno físico se convierte en una fuente constante de datos útiles para monitoreo, control y toma de decisiones.

Por su parte, el Internet Industrial de las Cosas (IIoT) lleva este concepto al ámbito productivo, donde los datos se utilizan para optimizar procesos, mejorar la eficiencia, garantizar la seguridad y permitir la automatización avanzada en sistemas industriales.

### A. Comparación conceptual

| Aspecto         | Internet Tradicional       | IoT                 | IIoT                      |
|-----------------|----------------------------|---------------------|---------------------------|
| Actor principal | Usuarios y servicios web   | Objetos + usuarios  | Máquinas y procesos       |
| Tipo de datos   | Multimedia                 | Variables físicas   | Datos de proceso          |
| Objetivo        | Intercambio de información | Monitoreo y control | Optimización y eficiencia |
| Ejemplo         | Redes sociales             | Casa inteligente    | Industria conectada       |

### B. IoT desde sistemas embebidos

Desde la perspectiva de sistemas embebidos, IoT se basa en la integración de cinco funciones:

- **Sensar:** capturar variables físicas (temperatura, presión, humedad, etc.)
- **Procesar:** filtrar y organizar los datos en el microcontrolador
- **Comunicar:** enviar información mediante protocolos y redes
- **Visualizar:** mostrar o almacenar los datos
- **Actuar:** ejecutar acciones sobre el sistema

Un sistema embebido deja de ser un dispositivo aislado y pasa a ser un nodo dentro de una arquitectura conectada.

### C. Arquitectura básica IoT

Una solución IoT se estructura en capas:

- **Capa física:** sensores, actuadores y microcontroladores
- **Capa de comunicación:** redes y tecnologías de transmisión
- **Capa de servicio:** almacenamiento, procesamiento y gestión de datos
- **Capa de aplicación:** interacción con el usuario y toma de decisiones

El valor del sistema no está únicamente en el nodo embebido, sino en la capacidad de transformar datos en información útil.

### D. Comunicación en IoT

Una solución IoT integra múltiples niveles de comunicación:

#### 1) Dentro del nodo:

- GPIO, ADC y PWM
- Protocolos como I2C, SPI y UART

#### 2) Hacia la red:

- Wi-Fi: alta tasa de datos
- BLE: bajo consumo
- LoRa: largo alcance
- Redes móviles: amplia cobertura

#### 3) Protocolos de aplicación:

- HTTP: simple pero menos eficiente
- MQTT: ligero y eficiente para IoT
- WebSocket: comunicación en tiempo real

### E. Lógica de implementación

El diseño de una solución IoT sigue una secuencia:

- 1) Definir la variable a medir
- 2) Seleccionar el sensor adecuado
- 3) Procesar los datos localmente
- 4) Transmitir la información
- 5) Almacenar o visualizar los datos
- 6) Generar una acción o decisión

### F. De IoT a IIoT

El IIoT aplica estos principios en entornos industriales, permitiendo:

- Mantenimiento predictivo
- Monitoreo de procesos
- Optimización energética
- Trazabilidad y control de calidad

En este contexto, los datos se convierten en un recurso clave para la toma de decisiones en tiempo real.

### G. Consideraciones de diseño

- Seguridad de la información
- Consumo energético
- Robustez del sistema
- Escalabilidad
- Mantenibilidad

## VI. PROYECTO FINAL DE SISTEMAS EMBEBIDOS

### A. Entregable 1

1) *Descripción del Problema:* En pequeños cultivos de hortalizas el riego y el control de condiciones ambientales suelen realizarse de forma manual...

#### 2) Objetivos:

a) *Objetivo General:* Diseñar e implementar un sistema embebido IoT para el monitoreo remoto y control automático...

#### b) Objetivos Específicos:

- Definir requisitos funcionales y no funcionales.
- Seleccionar tarjeta de desarrollo, sensores y actuadores.
- Implementar firmware modular e integrar comunicación IoT.

#### 3) Requisitos del Sistema:

a) *Funcionales:*

- Medición de temperatura, humedad de aire y sustrato.
- Control automático de bomba y actuador térmico con histéresis.
- Publicación de telemetría y recepción de comandos remotos.

b) *No Funcionales:*

- Uso de VS Code y PlatformIO (evitar .ino).
- Conectividad Wi-Fi con buffer local para desconexiones.
- Documentación técnica y commits frecuentes en GitHub.

4) *Diagrama General del Sistema:* La figura 8 muestra la arquitectura de alto nivel del sistema propuesto.

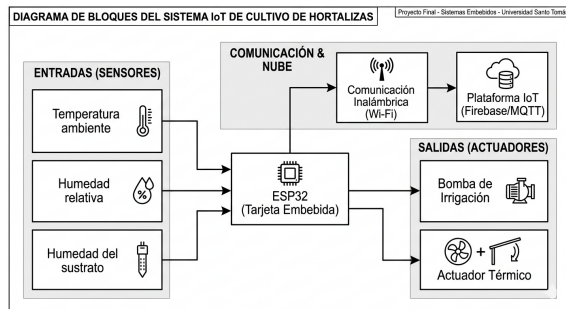


Figure 8. Diagrama general del sistema.

5) *Selección de Hardware y Diseño CAD: Tarjeta: ESP32.* Elegida por su conectividad Wi-Fi integrada y sus múltiples canales ADC. La figura 9 muestra el diseño CAD del sistema estructural básico.



Figure 9. CAD del sistema básico estructural.

6) *Lógica de Control:* El diagrama de flujo de la figura 10 describe el comportamiento del firmware embebido.

DIAGRAMA DE FLUJO DE FIRMWARE IoT EMBEBIDO

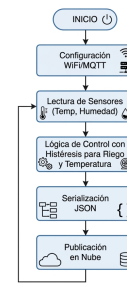


Figure 10. Diagrama de flujo del firmware.

B. *Entregable 2*

1) *Código Fuente:*

2) *Archivo README.md:*

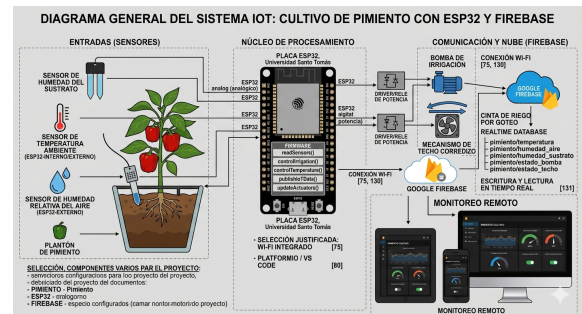


Figure 11. Diagrama general del sistema.

3) *Diagrama o imagen del sistema.:*

4) *Instrucciones de instalación.:* Para la correcta instalación y configuración del entorno de desarrollo se deben seguir los siguientes pasos:

- 1) **Instalación de Visual Studio Code:** Descargar e instalar Visual Studio Code desde la página oficial (<https://code.visualstudio.com/>).
- 2) **Instalación de la extensión PlatformIO:** Abrir Visual Studio Code, dirigirse al menú de extensiones y buscar PlatformIO IDE. Proceder con la instalación de la extensión.
- 3) **Creación de un nuevo proyecto:** En PlatformIO seleccionar la opción New Project, asignar un nombre al proyecto, elegir la placa correspondiente (ejemplo: Arduino Uno) y seleccionar el framework Arduino.
- 4) **Configuración del archivo platformio.ini:** Verificar que el archivo de configuración contenga las siguientes líneas:

```
[env:uno]
platform = atmelavr
board = uno
framework = arduino
```

- 5) **Estructura del proyecto:** Colocar el código fuente dentro de la carpeta src/ en el archivo main.cpp. Es importante incluir al inicio del archivo la librería principal:

```
#include <Arduino.h>
```

- 6) **Compilación y carga del programa:** Conectar la placa Arduino al ordenador, seleccionar el puerto correspondiente en PlatformIO y utilizar las opciones `Build` para compilar y `Upload` para cargar el programa en la placa.
- 7) **Monitoreo serial:** Para observar las lecturas del sensor y mensajes de depuración, abrir el `Serial Monitor` desde PlatformIO y configurar la velocidad en 9600 baudios.

Con estos pasos se garantiza que el entorno de desarrollo esté correctamente instalado y configurado, permitiendo la compilación y ejecución del sistema de riego automático basado en el sensor HW-080 y el control de relé.

5) *Instrucciones de ejecución.*: Una vez instalado y configurado el entorno de desarrollo, la ejecución del sistema se realiza siguiendo los pasos descritos a continuación:

- 1) **Conexión del hardware:** Conectar el sensor de humedad HW-080 al pin analógico configurado en el código, el módulo de relé al pin digital correspondiente y la bomba de agua a la salida del relé. Verificar que todas las conexiones estén firmes y correctas para evitar fallos eléctricos.
- 2) **Carga del programa en la placa:** Conectar la placa Arduino al ordenador mediante cable USB. En PlatformIO seleccionar la opción `Upload` para compilar y transferir el programa a la placa.
- 3) **Inicialización del sistema:** Una vez cargado el programa, la placa iniciará en estado seguro con el relé apagado. El sistema comenzará a leer periódicamente la humedad del suelo a través del sensor HW-080.
- 4) **Monitoreo de valores:** Abrir el `Serial Monitor` en PlatformIO con una velocidad de 9600 baudios. Observar las lecturas de humedad impresas en pantalla para verificar el correcto funcionamiento del sensor.
- 5) **Activación automática de la bomba:** Cuando el valor de humedad sea inferior al umbral definido en el código (`HUMEDAD_MIN`), el sistema activará automáticamente el relé y, por ende, la bomba de agua durante el tiempo establecido en la constante `TIEMPO_RIEGO`.
- 6) **Ciclo de operación:** Tras finalizar el riego, el sistema apagará la bomba y continuará monitoreando la humedad. Se respetará el intervalo mínimo entre riegos definido en el código para proteger la bomba y evitar sobreuso.

Con estos pasos, el sistema de riego automático ejecuta su ciclo de manera autónoma, garantizando que el cultivo de pimentón reciba la cantidad adecuada de agua según las condiciones de humedad del suelo.

6) *Descripción de librerías utilizadas.*: En el desarrollo del sistema de riego automático se emplearon las siguientes librerías y módulos del entorno Arduino:

- **Arduino.h:** Librería principal del entorno Arduino. Proporciona las funciones básicas necesarias para la programación del microcontrolador, como la configuración de

pines de entrada y salida, temporizadores, comunicación serial y estructuras fundamentales utilizadas en el desarrollo de aplicaciones embebidas.

- **WiFi.h:** Librería encargada de gestionar la conectividad inalámbrica del ESP32. Permite establecer la conexión a redes Wi-Fi, obtener información de la red, administrar direcciones IP y realizar comunicaciones con servidores a través de Internet.
- **Firestore\_Client.h:** Librería utilizada para la comunicación entre el ESP32 y la plataforma Firebase. Facilita la autenticación del dispositivo, el envío y recepción de datos en tiempo real, así como el acceso a bases de datos y servicios en la nube proporcionados por Firebase.
- **ESP32Servo.h:** Librería diseñada para el control de servomotores mediante el ESP32. Permite generar las señales PWM necesarias para posicionar el servo en ángulos específicos, simplificando su integración en aplicaciones de automatización y control.
- **ESP32PWM.h:** Librería que administra la generación de señales PWM (Pulse Width Modulation) en el ESP32. Ofrece funciones para configurar frecuencia, resolución y canales PWM, permitiendo controlar dispositivos como motores, LEDs y actuadores electrónicos.
- **DHT.h:** Librería utilizada para la adquisición de datos de sensores de temperatura y humedad de la familia DHT, como el DHT11 y DHT22. Proporciona funciones para inicializar el sensor y obtener mediciones de temperatura ambiente y humedad relativa de forma sencilla.

Estas librerías forman parte del núcleo estándar de Arduino y no requieren instalación adicional en PlatformIO, siempre que el proyecto esté configurado con `framework = arduino` en el archivo `platformio.ini`.

7) *Capturas de funcionamiento:* El funcionamiento del sistema está demostrado en el video establecido en el siguiente link: [https://youtube.com/shorts/z8qrX08eAyM?si=6lQUv-mnX7zXLUO\\_](https://youtube.com/shorts/z8qrX08eAyM?si=6lQUv-mnX7zXLUO_)

8) *Programación del ESP32:* La programación del ESP32 se desarrolló utilizando el entorno Arduino IDE, implementando una arquitectura basada en la adquisición de datos, comunicación con la nube y control de actuadores. El microcontrolador funciona como la unidad central del sistema, encargándose de la lectura de sensores, procesamiento de variables ambientales, ejecución de algoritmos de control y sincronización de información con Firebase.

Inicialmente, se incluyeron las librerías necesarias para el funcionamiento del sistema. Las librerías `WiFi.h` y `Firestore_Client.h` permiten la conexión a Internet y la comunicación con la base de datos en tiempo real. Por otra parte, `DHT.h` se utiliza para la lectura del sensor DHT11, mientras que `ESP32Servo.h` y `ESP32PWM.h` permiten el control del servomotor encargado de la apertura y cierre del techo automatizado.

Posteriormente, se definieron las credenciales de conexión a la red Wi-Fi y los parámetros de acceso a Firebase, incluyendo la clave de autenticación y la dirección de la base de datos. Asimismo, se establecieron los pines de conexión correspon-

dientes al sensor DHT11, sensor de humedad del suelo, relé de la bomba de agua y servomotor.

Dentro de la función `setup()`, el sistema realiza la inicialización de todos los periféricos. En esta etapa se configura la comunicación serial para monitoreo y depuración, se inicializa el sensor DHT11, se establece el estado inicial del relé de la bomba y se configura el servomotor mediante señales PWM. Posteriormente, el ESP32 se conecta a la red Wi-Fi y realiza la autenticación con Firebase para habilitar la comunicación con la base de datos.

Una característica importante de la implementación es el uso de *streams* de Firebase. Se configuraron dos canales de escucha permanentes asociados a las rutas `/control` y `/actuadores`. Estos canales permiten que cualquier cambio realizado desde la interfaz web sea recibido instantáneamente por el ESP32, posibilitando la operación remota del sistema sin necesidad de realizar consultas continuas a la base de datos.

Durante la ejecución continua del programa, implementada en la función `loop()`, se realiza la lectura de las variables ambientales. El sensor DHT11 proporciona los valores de temperatura y humedad relativa del aire, mientras que el sensor de humedad del suelo entrega una señal analógica que posteriormente es convertida a un porcentaje mediante una función de escalamiento.

Con las variables adquiridas se ejecuta la lógica de control automático del sistema. Para el techo automatizado se establecieron condiciones de apertura cuando la temperatura supera los 30 °C o la humedad relativa excede el 85%. De manera contraria, el techo se cierra cuando la temperatura desciende por debajo de 28.5 °C y la humedad relativa es inferior al 80%. Esta diferencia entre los umbrales de apertura y cierre implementa una banda de histéresis que evita conmutaciones repetitivas del actuador.

Para el sistema de riego, la bomba de agua es controlada mediante un relé. Cuando la humedad del suelo es inferior al 60%, la bomba se activa automáticamente para suministrar agua al cultivo. Una vez que la humedad supera el 75%, la bomba se desactiva, evitando el exceso de riego y optimizando el consumo de agua.

Finalmente, el ESP32 sincroniza periódicamente toda la información con Firebase. Cada tres segundos se envían los valores de temperatura, humedad del aire, humedad del suelo y lectura analógica del sensor. Asimismo, cuando el sistema opera en modo automático, también se actualiza el estado de los actuadores para que la interfaz web refleje en tiempo real las acciones ejecutadas por el controlador.

9) *Implementación de Firebase:* La plataforma Firebase Realtime Database fue utilizada como sistema de almacenamiento y sincronización de datos en la nube, permitiendo la comunicación bidireccional entre el ESP32 y la aplicación web. Esta implementación facilita la supervisión remota del sistema de cultivo y la actualización de información en tiempo real desde cualquier dispositivo con acceso a Internet.

La estructura de la base de datos fue organizada en diferentes nodos para separar las funciones de monitoreo, control y almacenamiento de información. El nodo `/sensores`

almacena las variables adquiridas por los sensores instalados en el sistema, incluyendo la temperatura ambiente, la humedad relativa del aire, la humedad del suelo y el valor analógico leído por el convertidor ADC del ESP32. Estas variables son actualizadas periódicamente por el microcontrolador y constituyen la principal fuente de información para el monitoreo del cultivo.

Por otra parte, el nodo `/actuadores` contiene el estado actual de los elementos de control del sistema. Dentro de este nodo se almacenan variables como el estado de la bomba de agua, el estado del techo automatizado y la posición del servomotor encargado de realizar la apertura y cierre de la estructura. Gracias a esta información, la interfaz web puede mostrar en tiempo real las acciones que está ejecutando el sistema.

El nodo `/control` fue diseñado para gestionar los modos de operación de los actuadores. En este apartado se almacenan las configuraciones correspondientes al modo automático y manual de la bomba de agua y del techo automatizado. Cuando el usuario modifica alguno de estos parámetros desde la aplicación web, la información es enviada a Firebase y posteriormente recibida por el ESP32 mediante mecanismos de escucha en tiempo real.

Adicionalmente, se implementó el nodo `/historial`, destinado al almacenamiento de registros históricos de las variables monitoreadas. Esta estructura permite conservar información relevante para futuros análisis del comportamiento ambiental del sistema y evaluar el desempeño del proceso de control implementado.

Para garantizar una comunicación eficiente entre la aplicación web y el microcontrolador, se emplearon dos *streams* de Firebase. El primero monitorea continuamente la ruta `/control`, permitiendo detectar cambios en los modos de operación seleccionados por el usuario. El segundo monitorea la ruta `/actuadores`, desde donde se reciben los comandos de activación y desactivación cuando el sistema opera en modo manual. Esta arquitectura permite que las acciones realizadas desde la interfaz web sean ejecutadas prácticamente en tiempo real por el ESP32.

Durante el funcionamiento normal del sistema, el ESP32 realiza actualizaciones periódicas de la base de datos cada tres segundos. En cada actualización se transmiten las variables medidas por los sensores y, cuando el sistema opera en modo automático, también se actualiza el estado de los actuadores. De esta manera, Firebase actúa como un intermediario entre el hardware y la interfaz de usuario, garantizando la sincronización continua de la información y proporcionando una solución robusta para el monitoreo y control remoto del cultivo inteligente.

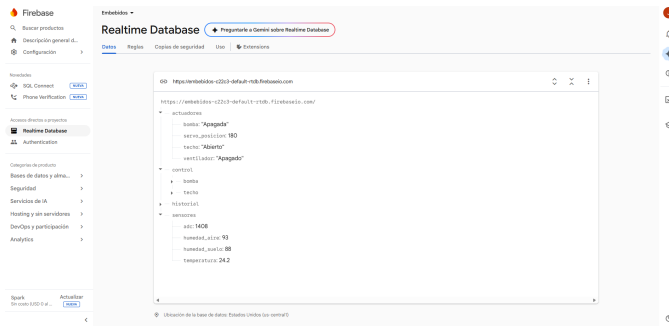


Figure 12. Estructura de la base de datos implementada en Firebase Realtime Database.

10) *Desarrollo de la aplicación web:* La aplicación web fue desarrollada utilizando tecnologías estándar de desarrollo web como HTML, CSS y JavaScript. Su objetivo principal es proporcionar una interfaz intuitiva para la supervisión y control remoto del sistema de cultivo inteligente.

La página permite visualizar en tiempo real las variables registradas por los sensores mediante indicadores numéricos y gráficos dinámicos. Para la representación gráfica de los datos históricos se empleó la librería Chart.js, la cual facilita el análisis del comportamiento de las variables ambientales a lo largo del tiempo.

La comunicación entre la aplicación web y Firebase se realiza mediante la integración de los servicios de la plataforma, permitiendo que cualquier actualización en la base de datos se refleje instantáneamente en la interfaz. De esta forma, el usuario puede monitorear el estado del sistema desde cualquier dispositivo con acceso a Internet.

Adicionalmente, la aplicación incorpora elementos de control que permiten modificar parámetros operativos y enviar comandos al ESP32. Esto proporciona una solución de monitoreo y control remoto centralizada, mejorando la accesibilidad y la gestión del sistema de cultivo inteligente.

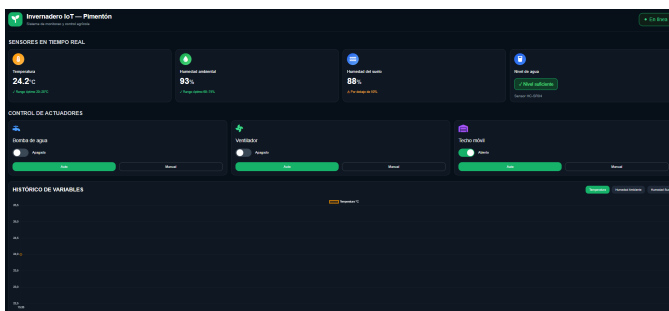


Figure 13. Muestra de la distribución final de la pagina

### 11) Historial de commits:

#### C. Pruebas de Sensores y Actuadores

1) *Prueba 1: Sensor DHT11:* **Objetivo:** Verificar el correcto funcionamiento del sensor DHT11 y validar la lectura de temperatura y humedad relativa del aire.

##### Metodología:

- 1) Se conectó el sensor DHT11 al ESP32 mediante la librería DHT.
- 2) Se realizaron lecturas periódicas cada tres segundos.
- 3) Los datos fueron visualizados mediante el monitor serial y enviados a Firebase.
- 4) Se acercó una fuente de calor moderada para observar cambios en la temperatura medida.
- 5) Se generó vapor de agua a corta distancia para evaluar la respuesta de la medición de humedad.

##### Resultados obtenidos:

- El sensor respondió adecuadamente a las variaciones de temperatura.
- Se observó un incremento gradual de la humedad relativa cuando se expuso al vapor de agua.
- Las mediciones permanecieron estables durante la operación continua.

**Conclusión:** El sensor DHT11 presentó un comportamiento adecuado para las necesidades del sistema de monitoreo ambiental.

2) *Prueba 2: Sensor de Humedad del Suelo:* **Objetivo:** Evaluar la capacidad del sensor para identificar distintos niveles de humedad presentes en el sustrato.

##### Metodología:

- 1) Se conectó el sensor a una entrada ADC del ESP32.
- 2) Se realizaron mediciones en tres condiciones:
  - Sensor en ambiente seco.
  - Sensor en suelo con humedad media.
  - Sensor en suelo completamente húmedo.
- 3) Los valores obtenidos fueron convertidos a porcentaje de humedad mediante la función de calibración implementada.

##### Resultados obtenidos:

- Los valores variaron de forma consistente con las condiciones del suelo.
- Se observó una clara diferencia entre suelo seco, húmedo y saturado.
- El sensor permitió establecer umbrales confiables para el control automático del riego.

**Conclusión:** El sensor de humedad del suelo permitió determinar correctamente el estado hídrico del cultivo y suministrar información confiable para la activación automática del sistema de riego.

3) *Prueba 3: Sistema de Riego Automático:* **Objetivo:** Verificar el funcionamiento de la bomba de agua y el relé de control.

##### Metodología:

- 1) Se simuló una condición de baja humedad del suelo.
- 2) Se observó la activación automática del relé.
- 3) Se verificó el funcionamiento de la bomba durante el tiempo programado.

##### Resultados obtenidos:

- La bomba se activó cuando la humedad fue inferior al umbral definido.

- El sistema desactivó automáticamente el riego al alcanzar el nivel adecuado.
- No se observaron activaciones erráticas durante las pruebas.

**Conclusión:** El sistema de riego cumplió satisfactoriamente con los criterios de control establecidos.

#### 4) Prueba 4: Sistema de Apertura Automática del Techo:

**Objetivo:** Evaluar la respuesta del servomotor encargado de la apertura y cierre del techo automatizado.

#### Metodología:

- 1) Se modificaron las variables ambientales para simular condiciones críticas.
- 2) Se verificó la apertura automática del techo.
- 3) Posteriormente se restablecieron las condiciones normales para observar el cierre automático.

#### Resultados obtenidos:

- El servomotor respondió correctamente a las órdenes de control.
- El techo se abrió y cerró según las condiciones programadas.
- La histéresis evitó movimientos repetitivos del actuador.

**Conclusión:** El sistema de ventilación automática funcionó correctamente durante todas las pruebas realizadas.

#### D. Problemas Encontrados Durante el Desarrollo

##### 1) Variabilidad en las lecturas del sensor de humedad:

Durante las primeras pruebas se observaron fluctuaciones en los valores obtenidos por el sensor de humedad del suelo. Estas variaciones fueron ocasionadas por diferencias en la compactación del sustrato, profundidad de inserción del sensor y ruido eléctrico presente en la señal analógica.

#### Solución implementada:

- Calibración experimental del sensor.
- Conversión de lecturas ADC a porcentaje de humedad.
- Implementación de histéresis en el algoritmo de control.

2) *Oscilaciones del servomotor:* Durante las pruebas iniciales el servomotor realizaba movimientos repetitivos cuando la temperatura se encontraba cerca del umbral de activación.

#### Solución implementada:

- Definición de umbrales independientes para apertura y cierre.
- Implementación de banda de histéresis.

3) *Pérdidas temporales de conexión Wi-Fi:* Se identificaron desconexiones ocasionales de la red inalámbrica que impedían la actualización inmediata de los datos en Firebase.

#### Solución implementada:

- Implementación de rutinas de reconexión automática.
- Verificación periódica del estado de la conexión.

##### 4) Retrasos en la sincronización de la aplicación web:

Las primeras versiones de la interfaz presentaban pequeños retardos entre la ejecución física y la actualización visual.

#### Solución implementada:

- Uso de Firebase Streams.
- Optimización de la estructura de la base de datos.

#### E. Lecciones Aprendidas

- 1) La calibración de sensores es indispensable para garantizar mediciones confiables.
- 2) La implementación de histéresis mejora significativamente la estabilidad de los sistemas de control.
- 3) La comunicación IoT requiere mecanismos de recuperación ante pérdidas de conectividad.
- 4) La modularización del software facilita el mantenimiento y la escalabilidad del sistema.
- 5) Firebase permite una sincronización eficiente entre hardware y aplicación web.
- 6) La validación individual de cada módulo simplifica la integración del sistema completo.
- 7) El desarrollo de soluciones IoT exige la integración coordinada de hardware, software y comunicaciones.

## VII. RESULTADOS

## VIII. CONCLUSIONES

En esta plantilla:

- Se mostró la estructura básica de un artículo IEEEtran en LaTeX en español. - Se incluyeron ejemplos mínimos de figuras, tablas, ecuaciones y citas. - Se proporcionaron comentarios en el código para guiar el uso en clase.

Trabajo futuro: reemplazar el contenido de ejemplo por el de tu proyecto, añadir más secciones si es necesario y ajustar el preámbulo según las normas de la conferencia o revista.

## AGRADECIMIENTOS

Se agradece a la institución X por el apoyo y a la agencia Y por la financiación del proyecto Z. (Sustituir por información real o eliminar esta sección si no aplica.)

## IX. ANEXOS

### A. Código Reto 1

```
//Microcontrolador: ESP32//

//DEFINICION DE PINES -//

const int boton1 = 25;
const int boton2 = 26;
const int boton3 = 27;

const int ledAcceso = 14;
const int ledError = 12;

const int releCerradura = 13;

//PARMETROS DEL SISTEMA //

const int secuenciaCorrecta[3] = {1,2,3};

const int maxIntentos = 3;

const unsigned long tiempoBloqueo = 10000;
const unsigned long tiempoApertura = 3000;

//VARIABLES DE CONTROL //

int secuenciaIngresada[3];
int indiceSecuencia = 0;

int contadorIntentos = 0;

bool sistemaBloqueado = false;

unsigned long inicioBloqueo = 0;
unsigned long inicioApertura = 0;

bool cerraduraAbierta = false;

//CONFIGURACION
```

```

void setup()
{
    pinMode(boton1, INPUT_PULLUP);
    pinMode(boton2, INPUT_PULLUP);
    pinMode(boton3, INPUT_PULLUP);

    pinMode(ledAcceso, OUTPUT);
    pinMode(ledError, OUTPUT);

    pinMode(releCerradura, OUTPUT);

    Serial.begin(115200);
}

//LOOP PRINCIPAL //

void loop()
{
    verificarBloqueo();

    if(!sistemaBloqueado)
    {
        leerBotones();
        verificarSecuencia();
    }

    controlarApertura();
}

//LECTURA DE BOTONES //

void leerBotones()
{
    if(digitalRead(boton1)==LOW)
    {
        registrarEntrada(1);
    }

    if(digitalRead(boton2)==LOW)
    {
        registrarEntrada(2);
    }

    if(digitalRead(boton3)==LOW)
    {
        registrarEntrada(3);
    }
}

// REGISTRAR ENTRADA //

void registrarEntrada(int valor)
{
    if(indiceSecuencia < 3)
    {
        secuenciaIngresada[indiceSecuencia] = valor;
        indiceSecuencia++;
    }
}

// VERIFICAR COMBINACION //

void verificarSecuencia()
{
    if(indiceSecuencia == 3)
    {
        if(combinacionCorrecta())
        {
            abrirCerradura();
            contadorIntentos = 0;
        }
        else
        {
            manejarIntentoFallido();
        }

        indiceSecuencia = 0;
    }
}

// COMPARACION //

bool combinacionCorrecta()
{
    for(int i=0; i<3; i++)
    {
        if(secuenciaIngresada[i] != secuenciaCorrecta[i])
            return false;
    }

    return true;
}

//APERTURA//

void abrirCerradura()
{
    digitalWrite(releCerradura, HIGH);
    digitalWrite(ledAcceso, HIGH);

    cerraduraAbierta = true;
    inicioApertura = millis();
}

```

```

//CONTROL APERTURA //

void controlarApertura()
{
    if(cerraduraAbierta)
    {
        if(millis() - inicioApertura > tiempoApertura)
        {
            digitalWrite(releCerradura, LOW);
            digitalWrite(ledAcceso, LOW);
            cerraduraAbierta = false;
        }
    }
}

//INTENTO FALLIDO//

void manejarIntentoFallido()
{
    contadorIntentos++;

    digitalWrite(ledError, HIGH);

    if(contadorIntentos >= maxIntentos)
    {
        sistemaBloqueado = true;
        inicioBloqueo = millis();
    }
}

// CONTROL BLOQUEO //

void verificarBloqueo()
{
    if(sistemaBloqueado)
    {
        if(millis() - inicioBloqueo > tiempoBloqueo)
        {
            sistemaBloqueado = false;
            contadorIntentos = 0;
            digitalWrite(ledError, LOW);
        }
    }
}

```

## B. Código Reto 2

```

const int sensorPin = 34;

const int ledNormal = 16;
const int ledAdvertencia = 17;
const int ledCritico = 18;

const int buzzer = 19;

/* Definición de umbrales del sistema */

float V1 = 30;
float V2 = 40;

float H = 2;

/* Definición de estados del sistema */

enum EstadoSistema
{
    NORMAL,
    ADVERTENCIA,
    CRITICO
};

EstadoSistema estadoActual = NORMAL;

/* Configuración inicial del sistema */

void setup()
{
    pinMode(ledNormal, OUTPUT);
    pinMode(ledAdvertencia, OUTPUT);
    pinMode(ledCritico, OUTPUT);

    pinMode(buzzer, OUTPUT);

    Serial.begin(115200);
}

/* Bucle principal de ejecución */

void loop()
{
    float valorSensor = leerSensor();

    actualizarEstado(valorSensor);

    actualizarSalidas();
}

/* Lectura del sensor analógico */

float leerSensor()
{
}

```

```

int lecturaADC = analogRead(sensorPin);

float temperatura = (lecturaADC/4095.0)*100;

return temperatura;
}

/* Actualizaci??n del estado del sistema con hist??resis */
void actualizarEstado(float valor)
{
    switch(estadoActual)
    {
        case NORMAL:

            if(valor > V1 + H)
                estadoActual = ADVERTENCIA;

            break;

        case ADVERTENCIA:

            if(valor > V2 + H)
                estadoActual = CRITICO;

            if(valor < V1 - H)
                estadoActual = NORMAL;

            break;

        case CRITICO:

            if(valor < V2 - H)
                estadoActual = ADVERTENCIA;

            break;

    }
}

/* Control de salidas del sistema */
void actualizarSalidas()
{
    digitalWrite(ledNormal,LOW);
    digitalWrite(ledAdvertencia,LOW);
    digitalWrite(ledCritico,LOW);

    switch(estadoActual)
    {
        case NORMAL:

            digitalWrite(ledNormal,HIGH);

            break;

        case ADVERTENCIA:

            digitalWrite(ledAdvertencia,HIGH);

            break;

        case CRITICO:

            digitalWrite(ledCritico,HIGH);
            digitalWrite(buzzer,HIGH);

            break;

    }
}

```

### C. Código Reto 3

```

const int sensorHumedad = 35;

const int releBomba = 23;

/* Defini??n de par??metros de control */
int umbralRiego = 40;

int hist = 5;

/* Par??metros de temporizaci??n del riego */
unsigned long tiempoRiego = 8000;

unsigned long inicioRiego = 0;

bool bombaActiva = false;

/* Configuraci??n inicial del sistema */
void setup()
{

```

```

    pinMode(releBomba,OUTPUT);
    Serial.begin(115200);
}

/* Bucle principal de ejecuci??n */

void loop()
{
    int humedad = leerHumedad();

    controlRiego(humedad);

    controlarTemporizador();
}

/* Lectura del sensor de humedad */
int leerHumedad()
{
    int lectura = analogRead(sensorHumedad);

    int porcentaje = map(lectura,0,4095,100,0);

    return porcentaje;
}

/* L??gica de control para iniciar el riego */
void controlRiego(int humedad)
{
    if(!bombaActiva)
    {
        if(humedad < (umbralRiego - hist))
        {
            iniciarRiego();
        }
    }
}

/* Activaci??n del sistema de riego */
void iniciarRiego()
{
    digitalWrite(releBomba,HIGH);

    bombaActiva = true;

    inicioRiego = millis();
}

/* Control del temporizador de riego */
void controlarTemporizador()
{
    if(bombaActiva)
    {
        if(millis() - inicioRiego > tiempoRiego)
        {
            detenerRiego();
        }
    }
}

/* Desactivaci??n del sistema de riego */
void detenerRiego()
{
    digitalWrite(releBomba,LOW);

    bombaActiva = false;
}

```